Dbt 02 Jinja Macros

Source originale : https://docs.getdbt.com/docs/build/jinja-macros

Jinja Template Designer Documentation

(external link)

dbt Jinja context

Macro properties

In dbt, you can combine SQL with

Jinja

, a templating language.

Using Jinja turns your dbt project into a programming environment for SQL, giving you the ability to do things that aren't normally possible in SQL. It's important to note that Jinja itself isn't a programming language; instead, it acts as a tool to enhance and extend the capabilities of SQL within your dbt projects.

For example, with Jinja, you can:

Use control structures (for example,

if

statements and

for

loops) in SQL

Use

environment variables

in your dbt project for production deployments

Change the way your project builds based on the current target.

Operate on the results of one query to generate another query, for example:

Return a list of payment methods, to create a subtotal column per payment method (pivot)

Return a list of columns in two relations, and select them in the same order to make it easier to union them together

Abstract snippets of SQL into reusable

macros

— these are analogous to functions in most programming languages.

If you've used the

{{ ref() }}

function

, you're already using Jinja!

Jinja can be used in any SQL in a dbt project, including

models

,
analyses
,
data tests
, and even
hooks
.
Ready to get started with Jinja and macros?
Check out the
tutorial on using Jinja
for a step-by-step example of using Jinja in a model, and turning it into a macro!
Here's an example of a dbt model that leverages Jinja:
/models/order_payment_method_amounts.sql
{
%
set
payment_methods
=
[
"bank_transfer"
,
"credit_card"
,
"gift_card"
]
%
}
select
order_id
,
{
%
for
payment_method
in
payment_methods

%
}
sum
(
case
when
payment_method
=
'{{payment_method}}'
then
amount
end
)
as
{{payment_method}}_amount
,
{
%
endfor
%
}
sum
(
amount
)
as
total_amount
from
app_data
.
payments
group
by
1

This query will get compiled to:

/models/order_payment_method_amounts.sql

```
select
order_id
,
sum
(
case
when
payment_method
=
'bank_transfer'
then
amount
end
)
as
bank_transfer_amount
,
sum
(
case
when
payment_method
=
'credit_card'
then
amount
end
)
as
credit_card_amount
,
sum
(
case
when
payment_method
```

=

'gift_card'

then

amount

end

)

as

gift_card_amount

,

sum

(

amount

)

as

total_amount

from

app_data

.

payments

group

by

1

You can recognize Jinja based on the delimiters the language uses, which we refer to as "curlies":

Expressions

{{ ... }}

: Expressions are used when you want to output a string. You can use expressions to reference

variables

and call

macros

.

Statements

{% ... %}

: Statements don't output a string. They are used for control flow, for example, to set up

for

loops and

if

statements, to

set

or

modify

variables, or to define macros.

Comments

{# ... #}

: Jinja comments are used to prevent the text within the comment from executing or outputing a string. Don't use

--

for comment.

When used in a dbt model, your Jinja needs to compile to a valid query. To check what SQL your Jinja compiles to:

Using

dbt

:

Click the compile button to see the compiled SQL in the Compiled SQL pane

Using

dbt v1

:

Run

dbt compile

from the command line. Then open the compiled SQL file in the

target/compiled/{project name}/

directory. Use a split screen in your code editor to keep both files open at once.

Macros

in Jinja are pieces of code that can be reused multiple times – they are analogous to "functions" in other programming languages, and are extremely useful if you find yourself repeating code across multiple models. Macros are defined in

.sql

files, typically in your

macros

directory (

docs

).

Macro files can contain one or more macros — here's an example:

macros/cents_to_dollars.sql

{
%
macro cents_to_dollars
(
column_name
,
scale
=
2
)
%
}
(
{{ column_name }}
/
100
)
::
numeric
(
16
,
{{ scale }}
)
{
%
endmacro
%
}

A model which uses this macro might look like:

models/stg_payments.sql

select
id
as
payment_id
,

{{ cents_to_dollars
(
'amount'
)
}}
as
amount_usd
,
.
from
app_data
.
payments
This would be
compiled
to:
target/compiled/models/stg_payments.sql
select
id
as
payment_id
,
(
amount
/
100
)
::
numeric
(
16
,
2
)
as
amount_usd

,

.

from

app_data

.

payments

■ Use Jinja's whitespace control to tidy your macros!

When you're modifying macros in your project, you might notice extra white space in your code in the

target/compiled

folder.

You can remove unwanted spaces and lines with Jinja's

whitespace control

by using a minus sign. For example, use

{{- ... -}}

or

{%- ... %}

around your macro definitions (such as

{%- macro generate_schema_name(...) -%} ... {%- endmacro -%}

).

A number of useful macros have also been grouped together into

packages

— our most popular package is

dbt-utils

.

After installing a package into your project, you can use any of the macros in your own project — make sure you qualify the macro by prefixing it with the

package name

:

select

field_1

,

field_2

,

field_3

,

field_4

,
field_5
,
count
(
*
)
from
my_table
{{ dbt_utils
.
dimensions
(
5
)
}}

You can also qualify a macro in your own project by prefixing it with your
package name
(this is mainly useful for package authors).

What parts of Jinja are dbt-specific?

There are certain expressions that are specific to dbt — these are documented in the
Jinja function reference
section of these docs. Further, docs blocks, snapshots, and
materializations
are custom Jinja
blocks
that exist only in dbt.

Which docs should I use when writing Jinja or creating a macro?

If you are stuck with a Jinja issue, it can get confusing where to check for more information. We recommend you check (in order):

Jinja's Template Designer Docs
: This is the best reference for most of the Jinja you'll use

Our Jinja function reference
: This documents any additional functionality we've added to Jinja in dbt.

Agate's table docs

: If you're operating on the result of a query, dbt will pass it back to you as an agate table. This means that the methods you call on the

table

belong to the Agate library rather than Jinja or dbt.

Why do I need to quote column names in Jinja?

In the

macro example

we passed the column name

amount

quotes:

{{ cents_to_dollars

(

'amount'

)

}}

as

amount_usd

We have to use quotes to pass the

string

'amount'

to the macro.

Without the quotes, the Jinja parser will look for a variable named

amount

. Since this doesn't exist, it will compile to nothing.

Quoting in Jinja can take a while to get used to! The rule is that you're within a Jinja expression or statement (i.e. within

{% ... %}

or

{{ ... }}

), you'll need to use quotes for any arguments that are strings.

Single and double quotes are equivalent in Jinja – just make sure you match them appropriately.

And if you do need to pass a variable as an argument, make sure you

don't nest your curlies

.

My compiled SQL has a lot of spaces and new lines, how can I get rid of it?

This is known as "whitespace control".

Use a minus sign (
-
, e.g.
{{- ... -}}
,
{%- ... %}
,
{#- ... -#}
) at the start or end of a block to strip whitespace before or after the block (more docs
here
). Check out the
tutorial on using Jinja
for an example.
Take caution: it's easy to fall down a rabbit hole when it comes to whitespace control!
How do I debug my Jinja?
You should get familiar with checking the compiled SQL in
target/compiled/<your_project>/
and the logs in
logs/dbt.log
to see what dbt is running behind the scenes.
You can also use the
log
function to debug Jinja by printing objects to the command line.
How do I document macros?
To document macros, use a
properties file
and nest the configurations under a
macros:
key
macros/properties.yml
macros
:
-
name
:
cents_to_dollars

description

:

A macro to convert cents to dollars

arguments

:

-

name

:

column_name

type

:

column

description

:

The name of the column you want to convert

-

name

:

precision

type

:

integer

description

:

Number of decimal places. Defaults to 2.

tip

From dbt v1.10, you can opt into validating the arguments you define in macro documentation using the

validate_macro_args

behavior change flag. When enabled, dbt will:

Infer arguments from the macro and includes them in the

manifest.json

file if no arguments are documented.

Raise a warning if documented argument names don't match the macro definition.

Raise a warning if

type

fields don't follow

supported formats

.

Learn more about

macro argument validation

.

When you create a

custom materialization

, dbt creates an associated macro with the following format:

materialization_{materialization_name}_{adapter}

To document a custom materialization, use the previously mentioned format to determine the associated macro name(s) to document.

macros/properties.yml

macros

:

-

name

:

materialization_my_materialization_name_default

description

:

A custom materialization to insert records into an append

-

only table and track when they were added.

-

name

:

materialization_my_materialization_name_xyz

description

:

A custom materialization to insert records into an append

-

only table and track when they were added.

Why does my dbt output have so many macros in it?

The output of a dbt run counts over 100 macros in your project!

$ dbt run

Running with

dbt

=

1.7

.0

Found

1

model,

0

tests,

0

snapshots,

0

analyses,

138

macros,

0

operations,

0

seed files,

0

sources

This is because dbt ships with its own project, which also includes macros! You can learn more about this

here

.

Just like well-written python is pythonic, well-written dbt code is dbtonic.

Once you learn the power of Jinja, it's common to want to abstract every repeated line into a macro! Remember that using Jinja can make your models harder for other users to interpret — we recommend favoring readability when mixing Jinja with SQL, even if it means repeating some lines of SQL in a few places. If all your models are macros, it might be worth re-assessing.

Writing a macro for the first time? Check whether we've open sourced one in

dbt-utils

that you can use, and save yourself some time!

{% set ... %}

can be used to create a new variable, or update an existing one. We recommend setting variables at the top of a model, rather than hardcoding it inline. This is a practice borrowed from many other coding languages, since it helps with readability, and comes in handy if you need to reference the variable in two places:

```
-- ■ This works, but can be hard to maintain as your code grows
{
%
for
payment_method
in
[
"bank_transfer"
,
"credit_card"
,
"gift_card"
]
%
}
.
{
%
endfor
%
}
-- ■ This is our preferred method of setting variables
{
%
set
payment_methods
=
[
"bank_transfer"
,
"credit_card"
,
"gift_card"
]
%
}
```

{
%
for
payment_method
in
payment_methods
%
}
.
{
%
endfor
%
}
0